Reporting Tool-Use DPO Under Fixed Budgets: Recipe–Checkpoint Profiles and Guardrail Trade-offs

Ilho Ahn
soleaf@gmail.com

June 2026

Abstract

Tool-use DPO is often reported as a scalar improvement over an SFT baseline, but structural function-call correctness, call-decision behavior, and instruction-following guardrails can move differently. We study source-native structural and behavior-oriented DPO negative recipes under a fixed pair, step, and reference protocol, including same-budget unfiltered controls, checkpoint-level variants, training-seed checks, and one independently reconstructed pair-pool replicate. The empirical core is a recipe–checkpoint profile rather than a single winning recipe. At the exploratory lower-regression early checkpoint, the clean structural recipe exceeds the clean behavior-oriented recipe by +3.33 points on BFCL core accuracy, but trails by 6.67 points on When2Call behavior accuracy and 5.31 points on When2Call macro F1. The source-axis sign pattern is preserved across two additional training seeds per main clean condition and in an independent pool-B replicate, where 12/12 pool-A and pool-B point-estimate sign checks match the expected direction. Because the behavior condition is task-family confounded with When2Call-style evaluation, and because the public When2Call slice used here has no direct-answer gold rows, we interpret these results as fixed-budget recipe comparisons over represented decision regimes, not as source-intrinsic superiority. Same-budget clean-vs-unfiltered effects are small or inconclusive, supporting semantic filtering as a pair-quality gate rather than as a universal downstream performance lever. Checkpoint choice also changes the guardrail profile: final checkpoints can retain or reduce intended gains while increasing IFEval prompt-strict regression. We therefore argue for reporting fixed-budget tool-use DPO as a profile: negative recipe, fixed budget, reference checkpoint, reported checkpoint, intended-axis gain, guardrail regression, seed and pair-sampling scope, data-quality gate, benchmark coverage, and overlap checks.

1 Introduction

Tool-augmented language models need to decide not only how to format a tool call, but also when to call a tool, when to ask a follow-up question, and when not to answer. Preference optimization for tool use therefore faces a multi-objective evaluation problem. Improvements on structural function-call metrics can coexist with regressions on behavior-control or instruction-following metrics, and a single aggregate score can hide that trade-off.

Rejected tool calls also vary in quality. Some rejected outputs are substantively incorrect negatives: malformed calls, wrong function choices, missing required arguments, or behaviorally inappropriate tool use. Others are semantically acceptable alternatives, optional/default/no-op differences, harmless normalizations, or ambiguous cases. Pair-quality control therefore needs to be separated from downstream metric movement: a cleaner pair set is not necessarily a better recipe for every tool-use metric.

We study this issue under a fixed-budget tool-use DPO setting, comparing source-native negative recipes and checkpoints against a shared SFT baseline. The central claim is not that one source is intrinsically superior or that filtering universally improves downstream performance. Rather, our evidence shows that a defensible tool-use DPO improvement claim should be reported as a recipe–checkpoint profile: the observed intended-axis movement, guardrail regression, seed scope, pair-sampling scope, benchmark coverage, and overlap checks jointly determine whether an apparent improvement is robust, axis-specific, or accompanied by regressions.

A key coverage boundary is visible before the main results: the public When2Call slice used here contains tool-call, follow-up-question, and unable-to-answer gold labels, but no direct-answer gold rows. We therefore treat direct-answer diagnostics as auxiliary coverage checks rather than as a matched behavior slice.

Our contributions are:

- A fixed-budget reporting profile for tool-use DPO that records recipe, budget, reference checkpoint, reported checkpoint, intended-axis metric, guardrail metric, seed scope, pair-sampling scope, data-quality gate, benchmark coverage, and overlap checks.
- An empirical recipe–checkpoint study under this profile, with the SFT reference, pair budget, optimizer-step budget, and DPO recipe fixed while comparing clean structural and behavior-oriented recipes, same-budget unfiltered controls, and early/final checkpoints.
- Evidence of axis-specific movement rather than source-intrinsic superiority: structural negatives are associated with stronger BFCL and function-calling movement, while behavior-oriented negatives are associated with stronger movement on the represented When2Call-style decision slice.
- Guardrail-aware checkpoint reporting showing that final checkpoints are not automatically preferred reporting points when IFEval prompt-strict regression is considered.
- Scoped robustness and claim-boundary checks: training-seed sign stability under fixed pair pools, one independent pair-pool replicate for the two main clean conditions, direct-answer coverage auditing, source-quality diagnostics, mixed-source sanity checking, and hash-based overlap checks.

2 Related Work

Function-calling evaluation. Function-calling benchmarks evaluate whether a model can select the right tool and produce valid arguments under a supplied schema. BFCL is the main structural evaluation surface in this study, covering function selection and tool-call correctness across multiple function-calling settings [11]; Yan [17] gives a broader function-calling perspective on scalable agent evaluation. Toolformer provides an early learning-based view of tool use as deciding when to call external tools, which tool to call, and how to incorporate tool results into generation [16]. Related tool-use and API-use benchmarks such as API-Bank and ToolLLM/ToolBench also motivate evaluating tool use as a structured prediction or tool-augmented reasoning problem rather than as ordinary free-form generation [6, 12].

Our structural axis uses BFCL-style function-calling metrics. We separate this axis from behavior-control metrics because a model can format and select functions correctly while still making poor decisions about whether a tool call is appropriate.

This separation is practical as well. BFCL emphasizes whether the model invokes the correct function with correct arguments in single-function, multiple-function, parallel-call, and live function-calling settings. ToolLLM/ToolBench and API-Bank broaden the API-use setting by adding tool retrieval, planning, and multi-step API interaction. These benchmarks make tool use measurable as structured action generation, but they do not by themselves settle whether a preference-training recipe should also improve call-decision behavior or instruction-following guardrails.

Call-decision and when-not-to-call evaluation. Behavior-oriented tool-use evaluation asks whether the model should call a tool, ask a follow-up question, abstain, or answer directly. When2Call is the closest benchmark family to this decision boundary: it evaluates not only tool-call generation, but also when a tool call should not be produced [15].

We therefore keep a separate behavior axis. We treat When2Call behavior accuracy and macro F1 as call-decision metrics, distinct from BFCL-style structural correctness. FunctionChat is used as a secondary transfer-oriented diagnostic rather than as part of the main reporting-profile claim.

When2Call explicitly separates tool-call, follow-up-question, unable-to-answer, and direct-answer decisions, making it complementary to function-call correctness. The public slice used in this study represents tool-call, follow-up-question, and unable-to-answer gold labels but not direct-answer gold labels; Appendix B

documents this coverage audit. FunctionChat-Bench similarly highlights that conversational tool-use evaluation extends beyond a single tool-call message: answer completion, slot questioning, and relevance detection can matter after or around the tool call [5]. We therefore keep these behavior and conversation diagnostics separate from the structural BFCL axis rather than collapsing them into a single score.

Preference optimization for tool-augmented models. DPO directly optimizes a policy from preference pairs without fitting a separate reward model, and has become a standard reference point for preference-based alignment [13]. DPO and related preference-optimization methods have also been applied to tool-augmented language models, including multi-turn settings such as DiaTool-DPO [4]. Our study does not introduce tool-use DPO or dialogue-level preference optimization.

This work is empirical and comparative: under a fixed pair budget and shared DPO recipe, we study how source-native negative recipes and checkpoint choice shift different evaluation axes, then use those shifts to motivate a reporting profile. This differs from work that proposes a new optimization objective, a new tool-use dataset, or a new tool-use benchmark.

Data quality, negative construction, and checkpoint selection. Tool-use preference data often mixes structural invalidity, semantic disagreement, behavior policy, and benchmark-domain effects. A single best-recipe narrative can be misleading when each negative source changes a different evaluation axis.

DPO checkpoints add a second source of variation, since the preference objective can be over-optimized relative to downstream metrics. Direct-alignment algorithms such as DPO can also exhibit over-optimization behavior as training progresses [14]. We therefore report early and final checkpoints together, including guardrail metrics, and treat checkpoint selection as part of the empirical result rather than as an implementation detail.

Prior work on DPO robustness and preference-data construction shows that noisy or ambiguous preference pairs can affect learned policies, and that data properties can matter independently of the optimizer [1, 10]. Tool-use negatives add another layer: a rejected sample can target wrong function selection, missing arguments, over-calling, under-calling, schema invalidity, or conversational policy. Cleaning can improve pair validity while still not guaranteeing movement on every downstream metric. Checkpoint selection creates a second quality-control problem, because optimizing longer against the preference objective can improve one intended axis while worsening an instruction-following guardrail.

Multi-metric reporting. The reporting-profile framing is closest in spirit to holistic evaluation work that treats language-model behavior as a multi-metric surface rather than as a single leaderboard score [7]. Our setting is narrower than HELM, but the same transparency issue appears: a tool-use DPO result can look improved on an intended tool-use axis while hiding checkpoint, guardrail, coverage, seed, or overlap qualifications.

3 Experimental Setup

3.1 Terminology and Scope

We use *negative recipe* for the procedure that constructs the rejected side of DPO preference pairs, including the source family and filtering gate. A *source-native* comparison keeps each recipe in its natural data family rather than forcing prompt-matched pairs across sources. A *fixed-budget* comparison fixes the pair count, optimizer-step budget, DPO hyperparameters, reference checkpoint, and reporting protocol. It does not claim exact equality in token counts or source distributions.

The *reported checkpoint* is the checkpoint selected for the main result after comparing checkpoint-level intended gains and guardrail regressions. A *guardrail metric* is not the primary intended axis; it is used to detect regressions, especially instruction-following degradation. A *recipe–checkpoint profile* is the full set of reported conditions and evidence: the negative recipe, fixed budget, reference and reported checkpoints, intended-axis gain, guardrail regression, seed scope, pair-sampling scope, data-quality gate, benchmark coverage, and overlap checks.

3.2 Baseline and Experimental Conditions

The baseline is a Qwen3-8B SFT checkpoint trained on a tool-use SFT mixture before DPO [18]. The SFT mixture combines function-calling and behavior-oriented tool-use examples: 70% xLAM/APIGen, 20% ToolACE, and 10% When2Call [8, 9, 15, 19]. This checkpoint establishes the shared reference policy. All DPO comparisons are evaluated against it, and the same checkpoint is used as the DPO reference.

Table 1 summarizes the main and robustness conditions used in this report. The pool-B replicate conditions are summarized in the data-sampling robustness table and are not plotted in Figure 1. Internal trace identifiers are kept in the accompanying repository for artifact lookup.

Table 1: Condition inventory.

Condition	Recipe family	Cleanliness	Preferred ckpt.	Pairs	Steps	Role
SFT baseline	SFT baseline	none	SFT best	0	0	reference baseline
clean structural	structural	clean	step50	3000	375	structural negative main condition
clean behavior	behavior	clean	step50	3000	375	behavior-oriented negative main condition
pool-B structural replicate	structural	clean pool-B	step50	3000	375	independent pair-pool replicate
pool-B behavior replicate	behavior	clean pool-B	step50	3000	375	independent pair-pool replicate
unfiltered structural	structural	unfiltered	step50	3000	375	same-budget structural control
unfiltered behavior	behavior	unfiltered	step50	3000	375	same-budget behavior control

All source-native DPO conditions use the same SFT reference checkpoint, beta 0.1, learning rate 5e-6, QLoRA DPO training stack, LoRA rank 16, effective batch size 8, 3000-pair budget, and 375 optimizer steps [2, 3]. Thus, fixed-budget means that the pair count, optimizer-step budget, DPO recipe, reference checkpoint, and reporting protocol are fixed. It does not mean that token counts are exactly equal across sources. Token-level accounting is tracked separately and should be read as a budget diagnostic, not as an equal-token causal guarantee.

We use "clean" for pairs that pass parser checks, schema checks, and semantic validity checks. Semantic validity asks whether the rejected output is verifiably worse than the chosen output under the provided tool schema and expected behavior. We use "unfiltered" for same-budget controls sampled before this filtering step. These controls test whether clean-vs-unfiltered status explains downstream movement better than the source-axis distinction.

3.3 Pair Construction and Filtering

Clean pair construction uses a high-confidence filtering pipeline rather than a parser-only filter. Candidate pairs must have valid tool schemas, parseable chosen and rejected outputs, stable prompt identifiers, and no obvious chosen/reference defect. Structural rows are retained when the rejected call is materially worse after canonicalization, such as a wrong function, missing required argument, wrong required argument value, wrong number of calls, or material type/schema error. Behavior rows are retained when the rejected response is behaviorally worse than the chosen response, including wrong-call, no-call, unnecessary follow-up, abstention, or answer-completion errors. Rows are dropped or quarantined when they are equivalent to the chosen output, ambiguous under the available schema, acceptable alternatives, malformed or non-strict in a non-informative way, or chosen/reference-suspect.

The two active negative recipes target different failure modes. The structural recipe starts from gold tool-call responses and constructs structural corruptions: wrong function names, dropped required arguments, altered required argument values, incorrect call counts, and material type or schema errors. The behavior-oriented recipe targets call-decision failures: calling when no call is appropriate, failing to call when a tool

is required, asking unnecessary follow-up questions, abstaining incorrectly, or producing answer-completion behavior inconsistent with the expected decision. The unfiltered controls use the same recipe families and pair budget, but they are sampled before the high-confidence semantic-clean gate and are subject only to trainability-oriented exclusions.

The final clean pools use LLM-assisted holdout gates with a closed-source judge snapshot. For each active clean source, at least 100 selected rows are audited under the same judge configuration. The release manifest records the snapshot identifier and request configuration, including JSON response format, `reasoning_effort=high`, `max_completion_tokens=4096`, and the fact that the temperature setting was omitted by the client, so the request used the OpenAI Chat Completions API default of `temperature=1`. A selected pool must reach at least 95% audit accept, at most 1% chosen/reference suspect rate, and no repeated reject-taxonomy bucket above 3%. These audits are quality-control gates, not human ground truth. The rubric rejects optional/default/no-op differences, harmless normalizations, acceptable alternate tools, semantic ambiguity, malformed or non-informative cases, and chosen/reference-suspect cases. The same-budget unfiltered controls bypass this clean gate and are included only to contextualize the clean-vs-unfiltered comparison.

3.4 Metrics

We evaluate intended capability axes separately from guardrail and diagnostic axes. The primary intended axes are BFCL core accuracy for structural function-calling behavior, and When2Call behavior accuracy and macro F1 for call-decision behavior. Guardrail and diagnostic axes include IFEval prompt-level strict accuracy and instruction-level accuracy [20], FunctionChat function-name accuracy as a secondary transfer-oriented tool-use check, strict parse success, SFT-dev tool-call accuracy, SFT-dev behavior accuracy, and nontrivial error rate.

The public release manifest fixes the benchmark snapshots, slices, sample counts, evaluator paths, parser policy, and released artifact paths summarized in Appendix A. These slices were frozen before comparing DPO variants and are reused unchanged across reported conditions. They are deterministic convenience slices for this fixed-budget comparison, not population-complete benchmark estimates. BFCL core accuracy is the frozen structural evaluator aggregate used for all DPO variants. When2Call behavior accuracy is computed on the frozen three-gold-label slice represented in the current manifest: tool-call, follow-up, and unable-to-answer. A coverage audit of the public `nvidia/When2Call` configurations found no direct-answer gold rows in the exposed labeled splits used for this study.

The current When2Call macro F1 follows the frozen evaluator implementation. It includes direct-answer as a zero-support class because the model can still predict direct-answer even though this slice has no direct-answer gold rows. This macro F1 should therefore be interpreted as the frozen evaluator's macro F1, not as a balanced four-label estimate of the full When2Call distribution. Strict parse success is reported separately; parser failures are not silently repaired into structural gains.

3.5 Reporting Profile

Table 2 states the reporting profile used in this paper. The profile is not intended as a universal standard. It is the reporting boundary required by the fixed-budget evidence here, and its purpose is to prevent an apparent tool-use DPO improvement from hiding recipe, checkpoint, guardrail, seed, coverage, or overlap qualifications.

Table 2: Fixed-budget tool-use DPO reporting profile used for the reported claims.

Profile field	Reported value in this study	Claim role
Negative recipe	clean structural and behavior-oriented recipes, plus same-budget unfiltered controls	separates recipe-associated movement from a single scalar ranking
Fixed budget	3000 pairs, 375 optimizer steps, beta 0.1, LR 5e-6, LoRA rank 16, effective batch 8	fixes pair/step/recipe controls without claiming equal loss-bearing tokens
Reference checkpoint	shared Qwen3-8B SFT checkpoint	avoids changing the DPO reference across recipes
Reported checkpoint	preferred step50 plus final checkpoint comparison	makes checkpoint selection part of the result rather than an implementation detail
Intended-axis metric	BFCL core for structural movement; When2Call accuracy/macro F1 for represented call-decision movement	prevents a single aggregate score from hiding axis-specific gains
Guardrail metric	IFEval prompt-level strict accuracy and instruction-level diagnostics	records instruction-following regression alongside intended gains
Seed scope	original seed plus two additional training seeds for the two main clean conditions	supports fixed-pool training-seed sign stability only
Pair-sampling scope	one independently reconstructed pool-B replicate for clean structural and behavior-oriented recipes	supports scoped data-sampling robustness for the two main clean conditions only
Data-quality gate	parser/schema checks, semantic validity checks, and closed-source LLM holdout gates for clean pools	treats filtering as quality control rather than a universal performance lever
Benchmark coverage	frozen BFCL, When2Call, SFT-dev, IFEval-style, and FunctionChat slices; direct-answer coverage audit	scopes behavior claims to represented regimes
Overlap checks	prompt ID, normalized user utterance, normalized tool schema, and combined user-plus-schema hashes	rules out exact overlap under reported keys without claiming semantic contamination-free generalization

3.6 Bootstrap and Matching Scope

For bootstrap uncertainty estimation, we group evaluation examples by `prompt_id` where the evaluation artifact exposes prompt identifiers. This grouping does not imply that the DPO training pools are prompt-matched across sources. The DPO training pools are source-native, and the behavior condition is task-family/domain-confounded with behavior-style data. The source-axis results should therefore be read as fixed-budget source-native recipe comparisons, not as causal source-intrinsic rankings.

Bootstrap intervals quantify evaluation-sample uncertainty only. They do not capture training-seed variance, DPO stochasticity, or data-sampling variance across independently reconstructed pair pools. For that reason, the follow-up training-seed check and pair-pool replicate are reported separately rather than merged with bootstrap intervals.

3.7 Matched Diagnostics

Matched diagnostics are used only to study source-quality risks. They summarize source-family differences in validity, ambiguity, and auditor disagreement under matched xLAM-style prompts. The final matched diagnostic audit uses a two-judge LLM workflow: local open-weight screening at temperature 0, followed by closed-source final adjudication over all 300 audit items. The released artifacts record the model identifiers and request settings. These diagnostics do not support downstream matched-DPO conclusions because the independent LLM-assisted audit criterion was not satisfied.

4 Results

4.1 Source-Native Recipes Populate Different Reporting Profiles

The clearest quantitative pattern is the source-axis comparison at the preferred step50 checkpoint. The preferred checkpoint is exploratory and selected after checkpoint analysis as the lower-regression reporting point for the current conditions, not as a pre-registered universal early-stopping rule. Table 3 summarizes source-axis differences at the preferred step50 checkpoint under source-native fixed-budget DPO. Positive deltas mean that the structural condition exceeds the behavior condition.

Confidence intervals are computed with grouped bootstrap over shared primary evaluation prompt IDs. The released primary manifest contains 1100 grouped prompt IDs per run pair across BFCL, When2Call, and SFT-dev auxiliary diagnostics: 300 BFCL examples, 300 When2Call examples, and 500 SFT-dev diagnostic examples. Each reported CI is recomputed on the metric-specific subset, so no Table 3 row should be

interpreted as using 1100 BFCL or 1100 When2Call examples. Each interval uses 1000 bootstrap iterations. The grouping is for uncertainty estimation and is separate from DPO training-pool matching.

Appendix E reports the corresponding absolute scores for the SFT baseline and reported checkpoints; the percentage-point deltas in the main tables should be read against those baseline values.

Table 3: Source-axis bootstrap results at the preferred step50 checkpoint. Deltas are percentage points.

Comparison	Metric	Delta	CI low	CI high	Interpretation
clean structural − clean behavior	BFCL core	+3.33	+1.53	+5.67	structural recipe has stronger BFCL movement
clean structural − clean behavior	W2C behavior acc.	-6.67	-10.75	-3.02	behavior-oriented recipe has stronger call-decision movement
clean structural − clean behavior	W2C macro F1	-5.31	-8.85	-2.34	behavior-oriented recipe has stronger macro-F1 movement
unfiltered structural − unfiltered behavior	BFCL core	+2.67	+1.02	+4.86	structural recipe has stronger BFCL movement
unfiltered structural − unfiltered behavior	W2C behavior acc.	-5.00	-8.70	-1.40	behavior-oriented recipe has stronger call-decision movement
unfiltered structural − unfiltered behavior	W2C macro F1	-4.21	-7.48	-1.23	behavior-oriented recipe has stronger macro-F1 movement

The pattern is consistent across clean and unfiltered same-budget comparisons. Structural negatives are associated with BFCL gains, while behavior-oriented negatives are associated with When2Call gains. BFCL core is not a pure structural-only construct: one category in the frozen slice covers relevance/irrelevance decisions, which overlap with when-not-to-call behavior. Appendix D therefore reports BFCL category deltas separately. The main interpretation is axis-specific gains with limited cross-axis transfer under source-native fixed-budget recipes. This motivates profile-based reporting over intended gains and guardrail regressions, but does not prove an intrinsic source ranking.

4.2 Clean-vs-Unfiltered Effects Are Weak or Inconclusive

Same-budget clean-vs-unfiltered comparisons are not the strongest explanation for downstream movement. At the preferred step50 checkpoint, clean structural minus unfiltered structural changes BFCL core accuracy by +0.33 points with CI95 [0.00, 1.08]. Clean behavior minus unfiltered behavior changes When2Call macro F1 by +0.55 points with CI95 [-0.83, 1.81].

In this fixed-budget setting, clean filtering, including semantic validity checks, is better interpreted as a quality-control mechanism than as a consistent downstream performance lever. Filtering may still reduce pair-level risk, but it is not the main empirical result here.

4.3 Additional Seeds Preserve the Source-Axis Direction

After the initial analysis, we ran a scoped training-seed check for the two main clean conditions. The pair pools, DPO recipe, reference checkpoint, and evaluation slices are fixed; only the DPO training seed varies. This check therefore measures training-seed sign stability for the source-axis comparison, not data-sampling robustness or source-intrinsic superiority.

Table 4 reports the source-axis sign check. Across the original seed, seed2, and seed3, the structural condition remains higher than the behavior condition on BFCL, while the behavior condition remains higher than the structural condition on When2Call behavior accuracy and macro F1. All 18 sign checks match the expected direction.

Table 4: Training-seed source-axis sign stability. Deltas are structural minus behavior.

Comparison	Checkpoint	BFCL delta	W2C acc. delta	W2C F1 delta
original	step50	+0.0333	-0.0667	-0.0531
original	best	+0.0333	-0.0633	-0.0483
seed2	step50	+0.0400	-0.0500	-0.0426
seed2	best	+0.0367	-0.0600	-0.0508
seed3	step50	+0.0300	-0.0633	-0.0489
seed3	best	+0.0300	-0.0600	-0.0506

For the two additional seeds, the selected adapter is the lowest-evaluation-loss adapter recorded by the fixed training recipe. At these selected adapters, the structural condition has mean BFCL 0.6967, When2Call behavior accuracy 0.5850, and When2Call macro F1 0.4828. The behavior condition has mean BFCL 0.6633, When2Call behavior accuracy 0.6450, and When2Call macro F1 0.5334. The observed seed-to-seed ranges are small in this scoped check, but the result should be read as sign-stability support rather than as a hierarchical uncertainty estimate.

4.4 An Independent Pair-Pool Replicate Preserves the Source-Axis Direction

The training-seed check keeps the original pool-A pairs fixed, so it does not test whether the source-axis pattern depends on the specific selected rows. We therefore ran a scoped pool-B replicate that independently reconstructs clean pair sets for the two main recipe families, reapplies the clean and closed-source LLM holdout gates, and then runs the same 3000-pair, 375-step DPO/evaluation recipe.

The accepted pool-B gates are strict enough that we treat this as a scoped robustness check rather than an uncontrolled regeneration. The structural pool-B construction selected 3000 rows from 9988 candidates, with 0 pair-ID overlap, 0 content-hash overlap, 401 prompt-ID overlaps with pool-A, closed-source holdout accept 0.97, and chosen/reference suspect rate 0.01. The behavior pool-B construction selected 3000 rows from 4294 candidates, with 0 pair-ID overlap, 0 content-hash overlap, 1337 prompt-ID overlaps, closed-source holdout accept 0.99, and chosen/reference suspect rate 0.01. Earlier behavior pool-B attempts failed the chosen/reference suspect gate and are not used for DPO. Internal trace identifiers for pool construction and DPO/evaluation runs are kept in the accompanying repository.

Table 5 compares the original pool-A results with the independent pool-B replicate. Across pool-A and pool-B, at step50 and best checkpoints, all 12 source-axis sign checks match the expected direction: the structural condition remains higher on BFCL, while the behavior condition remains higher on When2Call behavior accuracy and macro F1.

Table 5: Independent pair-pool source-axis sign stability. Deltas are structural minus behavior.

Comparison	Checkpoint	BFCL delta	W2C acc. delta	W2C F1 delta
pool-A original	step50	+0.0333	-0.0667	-0.0531
pool-A original	best	+0.0333	-0.0633	-0.0483
pool-B replicate	step50	+0.0333	-0.0633	-0.0504
pool-B replicate	best	+0.0400	-0.0667	-0.0583

These results support scoped data-sampling robustness for the two main clean source-native recipes. They do not imply broad data-distribution robustness, robustness for low-yield or mixed-source recipes, or source-intrinsic superiority.

4.5 Checkpoint Choice Changes the Guardrail Trade-off

The Pareto view compares intended-axis gain with IFEval prompt-strict regression. Figure 1 separates the two main intended metrics so that each panel compares the plotted conditions on a shared y-axis.

For the clean structural condition, step50 gives BFCL core delta +4.67 points with IFEval prompt-strict delta -6.25 points (IFEval CI95 [-11.46, -1.04]); the final checkpoint gives BFCL core delta +2.67 points

with IFEval prompt-strict delta -11.46 points (IFEval CI95 [-17.71, -5.21]). For the clean behavior condition, step50 gives When2Call macro F1 delta +4.86 points with IFEval prompt-strict delta -5.21 points (IFEval CI95 [-11.46, 0.00]); the final checkpoint gives When2Call macro F1 delta +4.68 points with IFEval prompt-strict delta -7.29 points (IFEval CI95 [-12.50, -2.08]). Absolute scores are reported in Appendix E; these deltas should be read against the SFT baseline values, including BFCL core 0.667, When2Call macro F1 0.481, and IFEval prompt-strict 0.635.

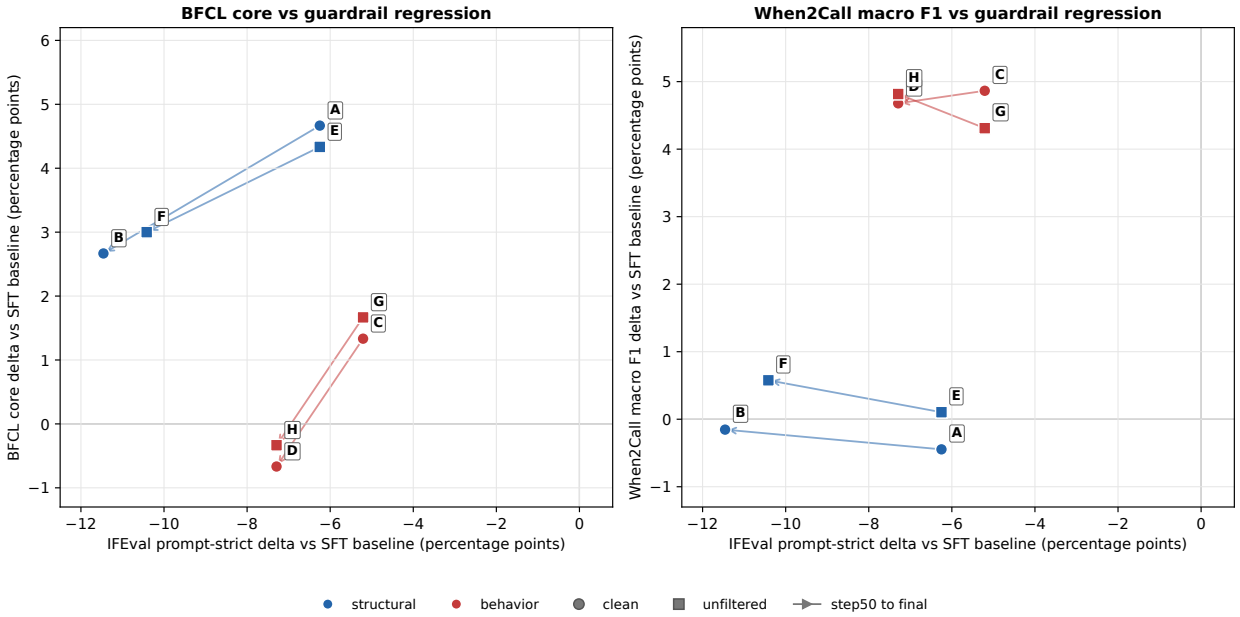

Figure 1: Pareto views of tool-use DPO under fixed budgets. Each panel plots intended-axis delta vs IFEval prompt-strict regression in percentage points. The left panel uses BFCL core accuracy as the intended metric, while the right panel uses When2Call macro F1. Arrows point from the preferred step50 checkpoint to the final checkpoint for the same condition. Positive y-values indicate intended-axis gains, and less negative x-values indicate smaller IFEval prompt-strict regression. Points closer to the upper right have larger intended-axis gains and smaller IFEval prompt-strict regressions. Point IDs: A/B = clean structural step50/final; C/D = clean behavior step50/final; E/F = unfiltered structural step50/final; G/H = unfiltered behavior step50/final. Full coordinates are kept with the figure artifacts in the accompanying repository.

Table 6 adds prompt-level bootstrap intervals for the IFEval guardrail deltas and the step50-vs-final differences. The structural rows give the clearest directional evidence that the early checkpoint has less prompt-strict regression than the final checkpoint, although their step50-vs-final intervals start at zero. The behavior rows move in the same direction, but their step50-vs-final intervals include negative values.

Table 6: IFEval prompt-strict bootstrap intervals. Deltas are percentage points. Positive step50-vs-final values mean the step50 checkpoint has higher prompt-strict accuracy than the final checkpoint.

Comparison	Condition	Delta	CI low	CI high	Groups
vs SFT	clean structural step50	-6.25	-11.46	-1.04	96
vs SFT	clean structural final	-11.46	-17.71	-5.21	96
vs SFT	clean behavior step50	-5.21	-11.46	+0.00	96
vs SFT	clean behavior final	-7.29	-12.50	-2.08	96
step50 − final	clean structural	+5.21	+0.00	+10.42	96
step50 − final	clean behavior	+2.08	-2.08	+6.25	96
step50 − final	unfiltered structural	+4.17	+0.00	+9.38	96
step50 − final	unfiltered behavior	+2.08	-2.08	+6.25	96

These comparisons make checkpoint-sensitive guardrail regression part of the reporting problem. They do not establish a universal rule that earlier checkpoints are always better. In the conditions studied here, however, final checkpoints can be less favorable reporting points than the preferred early checkpoint when intended gains and guardrail regressions are considered together. The arrows in Figure 1 show that moving from the early checkpoint to the final checkpoint can increase guardrail regression without proportional intended-axis gains. The unfiltered structural final checkpoint also shows a strict-parse diagnostic drop to 0.926, below the 0.963 SFT baseline, suggesting that over-training on this recipe can degrade parsing-related diagnostics even when BFCL core remains above baseline.

4.6 Post-Hoc Mixed-Source Sanity Check

A same-budget mixed-source run using 1500 clean structural pairs and 1500 clean behavior pairs improves both intended axes over the SFT baseline, but it does not dominate the single-source specialists and retains material IFEval regression. At step50, the mixed run reaches 0.700 BFCL core, below the structural specialist's 0.713, and 0.513 When2Call macro F1, below the behavior specialist's 0.530. Its IFEval prompt-strict score is 0.521, below both specialist step50 scores. We therefore treat the mixed run as another recipe profile within the same reporting framework, not as a generally best recipe or a Pareto-frontier improvement. Because this is a post-hoc sanity check, it is not included in Table 1's condition inventory; full values are kept in the accompanying repository.

4.7 Matched Diagnostics Remain Quality-Risk Evidence

Matched diagnostics suggest source-quality risks, especially for self and weak-model negatives. Because the independent LLM-assisted audit criterion was not satisfied, we do not report matched mini-DPO results. These diagnostics are therefore used as source-quality evidence and as a reason to keep matched downstream DPO out of the main claim, not as evidence for a source-intrinsic ranking. Appendix C reports the audit table.

5 Claim Scope

Table 7 summarizes which interpretations are supported by the reported evidence and which are outside the paper's scope.

Table 7: Claim scope matrix.

Claim	Status	Evidence	Limitation
Source-native negative recipes are associated with distinct evaluation axes	supported	source-axis step50 CIs exclude zero for BFCL and When2Call metrics	source-native rows do not prove source-intrinsic superiority
Training-seed stability for the main clean source-axis comparison	secondary	training-seed signs match the expected direction across the original seed plus two additional seeds per condition	fixed pair pools only; not data-sampling robustness or hierarchical uncertainty
Data-sampling robustness for the two main clean conditions	scoped	pool-B replicate preserves the source-axis sign pattern in 12/12 pool-A and pool-B point-estimate sign checks	one independent replicate only; not broad distribution robustness or source-intrinsic ranking
Tool-use DPO should be reported as a recipe–checkpoint profile	supported	intended-axis gains, IFEval regressions, checkpoint choice, seed scope, coverage, and overlap checks all affect interpretation	not a universal standard; supported here for fixed-budget evidence
Early checkpoint selection matters for guardrail-aware reporting	secondary	structural step50-vs-final IFEval intervals favor the early checkpoint; behavior intervals are directionally consistent but include zero	not every behavior-axis comparison gives a statistically strong checkpoint conclusion
Balanced direct-answer coverage in When2Call	not supported	coverage audit finds 0 direct-answer gold rows in the exposed public labeled configs used here	direct-answer behavior remains auxiliary rather than a matched When2Call slice
Broad clean-filter performance headline	not supported	clean-vs-unfiltered step50 effects are small or inconclusive	filtering remains useful as quality control
Mixed-source DPO is generally best	not supported	post-hoc mixed-source run improves both intended axes over SFT but trails the relevant specialists and retains IFEval regression	single-seed sanity check only; repository artifacts report full values
Matched diagnostics characterize source-quality risk	supported	matched source table and source-risk taxonomy	diagnostic only, not downstream matched-DPO evidence
Matched mini-DPO downstream result	not reported	independent audit criterion was not satisfied	require independent audit validation before downstream matched-DPO work

6 Discussion

The results are better viewed as a multi-objective reporting problem than as a scalar recipe ranking. The same fixed-budget DPO recipe can improve one tool-use axis while leaving another axis weaker or while increasing instruction-following regression. BFCL-style function-call correctness, When2Call-style call-decision behavior, FunctionChat-style transfer, and IFEval-style instruction-following therefore should not be collapsed into a single improvement headline without showing the profile behind it.

The source-axis pattern also changes how the negative recipes should be interpreted. In these experiments, structural negatives and behavior-oriented negatives should not be treated as universally ordered data sources; they are recipes that emphasize different downstream behaviors. This is why the main claim is a recipe–checkpoint profile rather than an intrinsic source ranking. The training-seed and pool-B checks strengthen the stability of this observed profile, but they do not turn the profile into a source-intrinsic causal result.

Checkpoint choice is part of the same reporting problem. Final checkpoints should not be treated as automatically representative without downstream metric checks, because longer preference optimization can change the guardrail trade-off without a proportional intended-axis gain. For this kind of tool-use DPO result, the reported checkpoint is therefore not merely a training detail; it is part of the empirical claim.

The analysis also separates two roles for data filtering. Filtering can still be valuable for quality control and risk reduction, especially when rejected responses may be equivalent, ambiguous, or chosen/reference-suspect. In this fixed-budget study, however, the clean-vs-unfiltered comparison is weaker than the source-axis separation between BFCL-oriented and When2Call-oriented movement. The practical implication is to report the data-quality gate as part of the evidence profile without converting it into a broad claim that clean filtering is always a downstream performance lever.

7 Limitations

The DPO training pools are source-native and not prompt-matched, so this paper does not establish source-intrinsic ranking. The behavior condition is also task-family/domain-confounded with When2Call-style evaluation. The fixed-budget design fixes pair count, optimizer steps, reference checkpoint, and recipe, but not token counts or source distributions. All reported DPO conditions also use one SFT reference model, one QLoRA-based DPO recipe, and one fixed pair/step budget, so results may differ for other models, objectives, adapter settings, or budgets.

The public When2Call slice used here does not provide direct-answer gold rows. We therefore scope behavior-axis claims to the represented tool-call, follow-up-question, and unable-to-answer regimes. Existing direct-answer diagnostics from SFT-dev and BFCL relevance rows are auxiliary only. A future matched behavior evaluation should explicitly label direct-answer cases rather than relying on auxiliary slices. Because independent audit did not validate the matched set at the required quality threshold, we also do not report matched mini-DPO results. LLM-assisted audit labels should be read as quality-control evidence rather than human annotations.

The robustness checks are intentionally scoped. The training-seed check covers only the two main clean conditions with fixed pair pools, and the pool-B replicate adds one independently reconstructed pair-pool check for the same two recipe families. Together, they support sign stability in this setting, not broad distribution robustness, robustness for mixed-source recipes, or a source-intrinsic causal ranking. The frozen evaluation slices are deterministic convenience slices, and bootstrap intervals cover evaluation-sample uncertainty rather than training stochasticity, data sampling, model choice, or benchmark construction. The small IFEval prompt-strict slice should be read as a guardrail diagnostic rather than a precise population estimate.

We do not study counterfactual schema-surface robustness under semantics-preserving changes to tool names, parameter names, schema descriptions, or tool-list composition. Hash-based overlap checks found no prompt-ID, normalized user-utterance, normalized tool-schema, or combined user-plus-schema overlap, but they do not rule out semantic near duplicates or upstream benchmark-family leakage.

8 Conclusion

This fixed-budget empirical study supports reporting tool-use DPO as a recipe–checkpoint profile rather than as a single recipe ranking. Source-native negative recipes are associated with distinct evaluation axes in this setting: structural negatives move BFCL and function-calling metrics more, while behavior-oriented negatives move the represented When2Call-style decision metrics more. The follow-up training-seed check preserves this source-axis sign pattern across additional training seeds for the two main clean conditions, and the pool-B replicate preserves it under one independent pair-pool reconstruction. At the same time, the results show limited cross-axis transfer and checkpoint-sensitive guardrail regression.

Practically, tool-use DPO reports should expose the recipe, checkpoint, intended metric, guardrail regression, seed scope, pair-sampling scope, benchmark coverage, data-quality gate, and overlap checks needed to interpret a reported gain. Under that profile, the evidence here supports a scoped fixed-budget recipe comparison, not universal filtering superiority, source-intrinsic ranking, broad distribution robustness, or a generally best mixed recipe.

9 Code and Data Availability

Code and artifacts are available at
`https://github.com/muted-color/tool-use-dpo-fixed-budget-report`, with the arXiv v1 artifact release fixed at tag `arxiv-v1` and commit `3f4799e`. The repository includes sanitized evaluation outputs, aggregate metrics, bootstrap artifacts, figure-generation scripts, DPO configuration files, condition and traceability inventory, benchmark version notes, overlap check artifacts, and reproduction commands. It is an artifact-level verification release, not an end-to-end retraining bundle.

Raw benchmark data, benchmark prompt text, full training examples, full processed DPO pairs, raw generated benchmark outputs, private audit materials, credentials, and model or adapter weights are not

redistributed. The repository instead provides scripts, hashes, sanitized outputs, aggregate metrics, and attribution notes where redistribution is constrained by license, benchmark-integrity, organizational, size, or provider-policy limits. Source-specific redistribution decisions are documented in the repository's data license and redistribution-policy notes.

A Evaluation Manifest

Table 8 summarizes the released reproducibility manifest. All evaluation slices were frozen before DPO-variant comparison and use deterministic decoding with `do_sample=false` and `enable_thinking=false`; the released manifest records frozen row hashes and sanitized per-example outputs rather than raw benchmark prompts or full generated outputs. Full artifact paths are listed in the repository manifest.

Table 8: Evaluation manifest for the public artifact release.

Surface	Source snapshot and slice	n	Evaluator and metric
BFCL	BFCL snapshot `61fc060`; first 60 rows each from simple, multiple, parallel, irrelevance, and live-multiple categories	300	`stage3d_eval.py` to `stage1_candidate_eval.py`; Qwen3 tool-call parser; exact-match BFCL core and category diagnostics
When2Call	`nvidia/When2Call@0582f77`; config `test`, split `mcq`; 100 tool-call, 100 follow-up, 100 unable, 0 direct-answer gold rows; coverage audit also finds 0 direct-answer gold rows in exposed labeled configs	300	`stage3d_eval.py` to `stage1_candidate_eval.py`; behavior accuracy over represented gold labels; macro F1 includes direct-answer as a zero-support evaluator class
SFT-dev aux.	Frozen SFT development diagnostics: 351 tool-call, 93 direct-answer, 30 follow-up, and 26 unable examples	500	`stage3d_eval.py` to `stage1_candidate_eval.py`; strict parse, tool-call accuracy, behavior accuracy, and nontrivial-error diagnostics
IFEval-style	`google/IFEval@966cd89`; first 100 raw rows from `ifeval_input_data.jsonl`; 96 prompt-level rows have at least one supported instruction	raw 100 / eval 96	`stage3d_eval.py` to `stage1_secondary_eval.py`; prompt-level strict denominator excludes zero-supported prompts; instruction-level accuracy uses supported instructions
FunctionChat	`kakao/FunctionChat-Bench@5ddb0b5`; 25 singlecall, 45 dialog, 30 call-decision rows; frozen row hashes match 100/100 rows for this commit	100	`stage3d_eval.py` to `stage1_secondary_eval.py`; function-name, argument, output-type, and hallucination diagnostics
Bootstrap	Sanitized primary and IFEval per-example outputs grouped by shared `prompt_id`; 1000 iterations; fixed script seeds	metric-specific	`scripts/compute_grouped_bootstrap.py` and `scripts/compute_ifeval_bootstrap.py`; percentile CI over paired metric deltas

Full commit identifiers and artifact paths are provided in `manifests/benchmark_manifest.yaml` and summarized in `docs/paper_manifest_summary.md`.

B Direct-Answer Coverage Audit

This audit checks the direct-answer coverage issue raised by the behavior-axis interpretation. It is a coverage and diagnostic check only; it does not add a new DPO run and does not turn the reported When2Call result into a balanced four-label estimate.

Table 9: Coverage audit of public `nvidia/When2Call` direct-answer gold rows.

Config	Split	Rows	Direct-answer gold status	Observed labeled gold values
`test`	`llm_judge`	300	0 labeled rows	100 tool-call, 100 follow-up, 100 unable
`test`	`mcq`	3652	0 labeled rows	1295 tool-call, 1062 follow-up, 1295 unable
`train_sft`	`train`	15000	unlabeled	public `correct_answer` is `None`
`train_pref`	`train`	9000	unlabeled	public `correct_answer` is `None`

The auxiliary diagnostics show that SFT-dev direct-answer rows remain high for the reported policies, while BFCL relevance direct-like rows are weak and often regressed relative to the SFT baseline. We therefore keep direct-answer behavior as an unresolved coverage limitation rather than using these diagnostics as a new main result.

Table 10: Auxiliary direct-answer diagnostics from existing artifacts. These rows are not a matched When2Call direct-answer slice. Values are accuracies.

Condition	Checkpoint	SFT-dev direct-answer	BFCL relevance direct-like	Combined
SFT baseline	SFT best	0.935	0.133	0.621
clean structural	step50	0.935	0.100	0.608
clean behavior	step50	0.935	0.050	0.588
additional structural seeds	best mean	0.978	0.042	0.611
additional behavior seeds	best mean	0.946	0.033	0.588

C Matched Diagnostic Source-Risk Table

Table 11 summarizes the source-risk taxonomy from the matched diagnostic audit. This table is separate from the clean-pool holdout gates described in Section 3.2; it covers matched xLAM-style diagnostic sources and is not a complete audit table for every active DPO training source. Because the independent LLM-assisted audit criterion was not satisfied, these diagnostics are quality-risk evidence only.

Table 11: Source-risk mechanism from matched diagnostic audit.

Source	Audited	Rejects	Reject rate	CI95	Main diagnostic reading
noised_gold	100	5	5%	[2.2, 11.2]%	low reject rate; mostly optional/default/no-op or not-verifiably-wrong cases
self	100	45	45%	[35.6, 54.8]%	high optional/default/no-op, ambiguity, and equivalence risk
weak_model	100	23	23%	[15.8, 32.2]%	moderate optional/default/no-op and not-verifiably-wrong risk, with a small acceptable-alternative component

D BFCL Category Breakdown

Table 12 separates the BFCL core aggregate into its frozen categories. The relevance/irrelevance bucket is included because it partially overlaps with call-decision behavior rather than only structural formatting. In these runs, behavior recipes do not outperform structural recipes on that bucket; the structural recipe is less regressed in relevance/irrelevance while also showing larger gains on live-multiple and multiple-call categories. This breakdown is diagnostic and should not be overread as a category-level causal claim.

Table 12: BFCL category deltas at the preferred step50 checkpoint. Values are percentage-point deltas versus the SFT baseline, with source-axis differences computed as structural minus behavior.

Category	Clean			Unfiltered		
	NG	Beh.	Diff.	NG	Beh.	Diff.
simple	-1.67	-1.67	+0.00	-1.67	-1.67	+0.00
multiple	+1.67	-1.67	+3.33	+1.67	-1.67	+3.33
parallel	+25.00	+25.00	+0.00	+25.00	+23.33	+1.67
relevance/irrelevance	-3.33	-8.33	+5.00	-3.33	-8.33	+5.00
live-multiple	+1.67	-6.67	+8.33	+0.00	-3.33	+3.33
BFCL core	+4.67	+1.33	+3.33	+4.33	+1.67	+2.67

E Absolute Scores at Reported Checkpoints

Table 13 reports absolute scores for the baseline, preferred step50 checkpoints, and final checkpoints. These values complement the delta-focused main text and help interpret whether a change occurs from a high or low baseline.

Table 13: Absolute metrics for the SFT baseline and reported DPO checkpoints.

Condition	Checkpoint	BFCL core	W2C acc.	W2C macro F1	IFEval prompt strict	Strict parse
SFT baseline	SFT best	0.667	0.573	0.481	0.635	0.963
clean structural	step50	0.713	0.567	0.477	0.573	0.991
clean structural	final	0.693	0.577	0.479	0.521	0.972
clean behavior	step50	0.680	0.633	0.530	0.583	0.991
clean behavior	final	0.660	0.640	0.528	0.562	0.991
unfiltered structural	step50	0.710	0.577	0.482	0.573	0.991
unfiltered structural	final	0.697	0.593	0.487	0.531	0.926
unfiltered behavior	step50	0.683	0.627	0.524	0.583	0.991
unfiltered behavior	final	0.663	0.640	0.529	0.562	0.991

References

[1] Sayak Ray Chowdhury, Anush Kini, and Nagarajan Natarajan. Provably robust DPO: Aligning language models with noisy feedback. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 42258–42274, 2024.

[2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[4] Sunghee Jung, Donghun Lee, Shinbok Lee, Gaeun Seo, Daniel Lee, Byeongil Ko, Junrae Cho, Kihyun Kim, EungGyun Kim, and Myeongcheol Shin. DiaTool-DPO: Multi-turn direct preference optimization for tool-augmented large language models. In *Proceedings of the 26th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pages 397–416, 2025.

[5] Shinbok Lee, Gaeun Seo, Daniel Lee, Byeongil Ko, Sunghee Jung, and Myeongcheol Shin. FunctionChat-Bench: Comprehensive evaluation of language models' generative capabilities in korean tool-use dialogs. *arXiv preprint arXiv:2411.14054*, 2024.

[6] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187.

[7] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023.

[8] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, Zezhong Wang, Yuxian Wang, Wu Ning, Yutai Hou, Bin Wang, Chuhan Wu, Xinzhi Wang, Yong Liu, Yasheng Wang, Duyu Tang, Dandan Tu, Lifeng Shang, Xin Jiang, Ruiming Tang, Defu Lian, Qun Liu, and Enhong Chen. ToolACE: Winning the points of LLM function calling. *arXiv preprint arXiv:2409.00920*, 2024.

[9] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh Murthy, Liangwei Yang, Silvio Savarese, Juan Carlos Niebles, Huan

Wang, Shelby Heinecke, and Caiming Xiong. APIGen: Automated pipeline for generating verifiable and diverse function-calling datasets. In *Advances in Neural Information Processing Systems*, 2024.

[10] Yu Pan, Zhongze Cai, Guanting Chen, Huaiyang Zhong, and Chonghuan Wang. What matters in data for DPO? *arXiv preprint arXiv:2508.18312*, 2025.

[11] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392, 2025.

[12] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*, 2024. Spotlight. arXiv:2307.16789.

[13] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

[14] Rafael Rafailov, Yaswanth Chittepu, Ryan Park, Harshit Sikchi, Joey Hejna, W. Bradley Knox, Chelsea Finn, and Scott Niekum. Scaling laws for reward model overoptimization in direct alignment algorithms. *arXiv preprint arXiv:2406.02900*, 2024.

[15] Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. When2Call: When (not) to call tools. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3391–3409, 2025. doi: 10.18653/v1/2025.naacl-long.174.

[16] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

[17] Fanjia Yan. A function calling perspective on scalable large language model agent evaluation. Master's thesis, EECS Department, University of California, Berkeley, 2025.

[18] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[19] Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, Weiran Yao, Juntao Tan, Zhiwei Liu, Yihao Feng, Juan Carlos Niebles, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. xLAM: A family of large action models to empower AI agent systems. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2025. arXiv:2409.03215.

[20] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.